

HDOC DAY – 24

ALGORITHM:

Step 1: Start.

Step 2: Declare choice as an integer.

Step 3: Display the menu

Step 4: Read the user's choice.

Step 5: Use a switch statement.

Step 6: According to the choice:

- Case 1: Call pattern1().
 - Input number of rows n.
 - Use nested loops.
 - Print * exactly i times in row i.
- Case 2: Call pattern2().
 - Input n.
 - In row i, print the number i, i times.
- Case 3: Call pattern3().
 - Input n.
 - In row i, print natural numbers from 1 to i.
- Case 4: Call pattern4().
 - Input n.
 - Initialize num = 1.
 - Print consecutive numbers using nested loops and increment num after each number.
- Case 5: Call pattern5().
 - Input n.
 - Check that $1 \leq n \leq 26$.

- Print letters from A up to the corresponding letter in each row.
- Case 6: Exit the program.
- Default: Display "Invalid choice!".

Step 7: Continue displaying the menu until the user selects Exit.

Step 8: Stop.

C PROGRAM:

```
#include <stdio.h>
```

```
/* Function declarations */
```

```
void pattern1(void);
```

```
void pattern2(void);
```

```
void pattern3(void);
```

```
void pattern4(void);
```

```
void pattern5(void);
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    do
```

```
    {
```

```
        printf("\n--- Increasing Patterns Menu ---\n");
```

```
        printf("1. Increasing right-angled triangle\n");
```

```
        printf("2. Increasing row-number triangle\n");
```

```
        printf("3. Increasing natural-number pattern\n");
```

```
        printf("4. Consecutive-number triangle\n");
```

```
        printf("5. Increasing alphabet pattern\n");
```

```
        printf("6. Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);  
  
switch(choice)  
{  
    case 1:  
        pattern1();  
        break;  
  
    case 2:  
        pattern2();  
        break;  
  
    case 3:  
        pattern3();  
        break;  
  
    case 4:  
        pattern4();  
        break;  
  
    case 5:  
        pattern5();  
        break;  
  
    case 6:  
        printf("Exiting program...\n");
```

```

        break;

    default:
        printf("Invalid choice!\n");
    }

} while(choice != 6);

return 0;
}

/* 1. Increasing right-angled triangle */
void pattern1(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if(n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for(i = 1; i <= n; i++)
    {

```

```
    for(j = 1; j <= i; j++)  
    {  
        printf("* ");  
    }  
    printf("\n");  
}  
}
```

/* 2. Increasing row-number triangle */

```
void pattern2(void)  
{  
    int n, i, j;  
  
    printf("Enter the number of rows: ");  
    scanf("%d", &n);  
  
    if(n <= 0)  
    {  
        printf("Number of rows must be positive.\n");  
        return;  
    }  
  
    for(i = 1; i <= n; i++)  
    {  
        for(j = 1; j <= i; j++)  
        {  
            printf("%d ", i);
```

```
    }  
    printf("\n");  
}  
}
```

/* 3. Increasing natural-number pattern */

```
void pattern3(void)  
{  
    int n, i, j;  
  
    printf("Enter the number of rows: ");  
    scanf("%d", &n);  
  
    if(n <= 0)  
    {  
        printf("Number of rows must be positive.\n");  
        return;  
    }  
  
    for(i = 1; i <= n; i++)  
    {  
        for(j = 1; j <= i; j++)  
        {  
            printf("%d ", j);  
        }  
        printf("\n");  
    }  
}
```

```
}
```

```
/* 4. Consecutive-number triangle */
```

```
void pattern4(void)
```

```
{
```

```
    int n, i, j, num = 1;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &n);
```

```
    if(n <= 0)
```

```
    {
```

```
        printf("Number of rows must be positive.\n");
```

```
        return;
```

```
    }
```

```
    for(i = 1; i <= n; i++)
```

```
    {
```

```
        for(j = 1; j <= i; j++)
```

```
        {
```

```
            printf("%d ", num);
```

```
            num++;
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

```

/* 5. Increasing alphabet pattern */
void pattern5(void)
{
    int n, i, j;

    printf("Enter the number of rows (1-26): ");
    scanf("%d", &n);

    if(n < 1 || n > 26)
    {
        printf("Number of rows must be between 1 and 26.\n");
        return;
    }

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= i; j++)
        {
            printf("%c ", 'A' + j - 1);
        }
        printf("\n");
    }
}

```

TEST CASE TABLE:

TEST CASE	CHOICE	INPUT(n)	EXPECTED RESULT
1	1	4	Prints 4-row increasing * triangle

2	2	4	Prints 1, 2 2, 3 3 3, 4 4 4 4
3	3	4	Prints natural numbers from 1 to row number
4	4	4	Prints consecutive numbers 1 through 10 triangularly
5	5	4	Prints A, A B, A B C, A B C D
6	5	27	Displays invalid rows message
7	3	0	Displays invalid rows message
8	8	--	Displays "Invalid choice!"
9	6	--	Exits the program

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 1

Enter the number of rows: 4

```
*
* *
* * *
* * * *
```

— Increasing Patterns Menu —

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 2

Enter the number of rows: 4

```
1
2 2
3 3 3
4 4 4 4
```

— Increasing Patterns Menu —

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 3

Enter the number of rows: 4

```
1
1 2
1 2 3
1 2 3 4
```

— Increasing Patterns Menu —

1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit

Enter your choice: 4

Enter the number of rows: 4

```
1
```